

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E MATEMATICA APPLICATA



SOFTWARE ARCHITECTURE DESIGN

Music Playlist Manager

PROJECT WORK

Third delivery: 3rd Sprint Release

Group members:

Cognome	Nome	Matricola
Amoroso	Chiara	IE22700309
Contaldo	Luigi	IE22700111
D'Errico	Fabrizio	IE22700121
Esposito	Antonio	IE22700003

Anno accademico 2025/2026

Sommario

Third Sprint Backlog.....	3
Update Product Backlog.....	7
Project Burndown Chart.....	13
Analisi del Burndown Chart.....	13
Sprint Review Report.....	14
Stato delle user stories	14
Sprint Retrospective Report.....	15
Funzionalità Aggiuntive	17
Principi di Buon Design e Pattern Adottati.....	18
Pattern Strategy - ModalitaRiproduzione.....	18
Pattern Singleton - Catalogo e CatalogoPlaylist	18
Pattern Command - Operazione e GestoreOperazioni	18
Pattern Observer - ObservableList	18
Separation of Concerns.....	18
DRY - Don't Repeat Yourself	18
Single Responsibility Principle.....	19
YAGNI - You Aren't Gonna Need It.....	19

Third Sprint Backlog

La tabella riportata di seguito rappresenta il backlog della 3rd Sprint, durante la quale il team ha completato l'implementazione delle funzionalità rimanenti dell'applicazione. Ogni voce della tabella corrisponde a una user story estratta dal Product Backlog e descrive, in modo sintetico ma completo, le attività necessarie per soddisfare il requisito funzionale espresso dal punto di vista dell'utente finale. Per ciascuna user story sono stati definiti i relativi task operativi. A ciascuna voce è inoltre stato assegnato un valore in Story Points (SP), utile per stimare l'impegno richiesto e monitorare la velocità del team.

ID	ITEM	ITEM TASKS	SP
14	Come utente voglio annullare l'ultima operazione di aggiunta o rimozione di una traccia da una playlist, così da correggere errori accidentali senza doverli rifare manualmente.	Task 14.1 : Creare la classe astratta Operazione con metodi astratti esegui() e annulla() e un metodo getDescrizione() per mostrare l'operazione nella UI Task 14.2 : Implementare AggiungiTracciaCatalogo e RimuoviTracciaCatalogo come sottoclassi di Operazione. La RimuoviTracciaCatalogo deve memorizzare anche le playlist da cui la traccia è stata rimossa, per ripristinarla correttamente nell'undo Task 14.3: Implementare aggiungiTracciaPlaylist e RimuoviTracciaPlaylist come sottoclassi di Operazione. La RimuoviTracciaPlaylist deve memorizzare l'indice originale della traccia per reinserirla nella posizione corretta Task 14.4: Creare GestoreOperazioni con stack LIFO di dimensione massima 10 (rimuove l'operazione più vecchia se si supera il limite) e metodi eseguiOperazione(Operazione op) e annullaUltimaOperazione(). Task 14.5: Modificare PrimaryController per usare GestoreOperazioni.eseguiOperazione() invece di chiamare direttamente i metodi di Libreria e Playlist. Task 14.6: Aggiungere pulsante "Annulla" nella UI che mostra la descrizione dell'ultima operazione effettuata e si disabilita automaticamente quando lo stack è vuoto. Task 14.7: Scrivere test JUnit per: undo di aggiunta, undo di rimozione (con verifica della posizione), undo multipli consecutivi, limite massimo di 10 livelli.	3

15	Come utente voglio vedere nella home page le tracce e le playlist riprodotte più frequentemente, così da ritrovare subito i miei contenuti preferiti.	Task 15.1: Aggiungere un attributo <code>contaRiproduzioni (int)</code> alla classe <code>Traccia</code> con relativi getter e metodo <code>incrementaRiproduzioni()</code>. Task 15.2: Aggiungere un attributo <code>contaRiproduzioni (int)</code> alla classe <code>Playlist</code> con relativi getter e metodo <code>incrementaRiproduzioni()</code>. Task 15.3: Incrementare il contatore in <code>Lettore</code> al completamento simulato di ogni traccia (quando il timer raggiunge la durata totale) Task 15.4: Implementare <code>Libreria.getTopTracce(int n)</code> che restituisce le top N tracce ordinate per <code>contaRiproduzioni</code> decrescente. Task 15.5: Implementare <code>Libreria.getTopPlaylist(int n)</code> che restituisce le top N playlist ordinate per <code>contaRiproduzioni</code> decrescente. Task 15.6: Creare nel file <code>.fxml</code> la schermata home con due sezioni: "Top 5 Tracce" e "Top 5 Playlist", con label numero di riproduzioni accanto a ogni elemento. Task 15.7: Implementare in <code>PrimaryController</code> l'aggiornamento automatico delle liste top tramite binding con la <code>Libreria</code>. Task 15.8: Scrivere test JUnit per <code>getTopTracce()</code> e <code>getTopPlaylist()</code> verificando ordinamento corretto e gestione di liste vuote	3
18	Come utente voglio generare automaticamente una playlist per tag visivo con le tracce corrispondenti, così da organizzare rapidamente i brani contrassegnati.	Task 18.1: Implementazione del metodo <code>filtraPerTag</code> nella classe <code>GeneratorePlaylistAutomatica</code>. Task 18.2: Implementazione del metodo <code>nomePlaylistAutomatica</code> nella classe <code>GeneratorePlaylistAutomatica</code>. Task 18.3: Implementazione dei metodi <code>generaPerTag</code> nella classe <code>GeneratorePlaylistAutomatica</code>. Task 18.4: Implementazione della classe di test <code>GeneratorePlaylistAutomaticaTest</code> per il caso dei tag.	3
19	Come utente voglio poter cambiare l'ordine delle tracce	Task 19.1: Implementare i metodi <code>Playlist.spostaSu(Traccia t)</code> e	3

	all'interno di una playlist affinché io possa personalizzare la sequenza di ascolto.	Playlist.spostaGiu(Traccia t) sulla ObservableList<Traccia> per spostare la traccia di una posizione. Task 19.2: Aggiungere pulsanti "▲" e "▼" nella GUI per consentire il riordino manuale. Task 19.3: Collegare i pulsanti di riordino al PrimaryController che invoca spostaSu() o spostaGiu() sulla Playlist corrente. Task 19.4: Garantire che il riordino funzioni anche durante la riproduzione: aggiornare dinamicamente la Queue<Traccia> del Lettore senza interrompere il playback. Task 19.5: Scrivere test JUnit per spostaSu() e spostaGiu() verificando i casi: primo elemento (spostaSu non fa nulla), ultimo elemento (spostaGiu non fa nulla), elemento centrale.	
20	Come utente voglio eliminare una playlist dal catalogo playlist, così da rimuovere raccolte che non voglio più mantenere.	Task 20.1: Aggiornare il bottone "Rimuovi" in PrimaryController per gestire in modo contestuale la rimozione di una traccia dalla playlist, l'eliminazione della playlist stessa, la rimozione dal catalogo generale e il caso in cui nessuna selezione è attiva. Task 20.2: Mostrare un dialog di conferma prima di procedere con l'eliminazione della playlist, informando l'utente che le tracce rimarranno nel catalogo generale. Task 20.3: Gestire il caso in cui la playlist eliminata sia quella attualmente in riproduzione, fermando il lettore e resettando lo stato del player. Task 20.4: Scrivere test JUnit per verificare la corretta rimozione della playlist, la persistenza delle tracce nel catalogo generale e i casi limite come nome inesistente e catalogo vuoto dopo eliminazione.	2
21	Come utente voglio salvare tracce e playlist, così da mantenere i dati inseriti anche dopo la chiusura dell'applicazione.	Task 21.1: Aggiungere la logica per convertire le classi Traccia e Playlist in formato JSON e viceversa. Assicurarsi che tutti i campi vengano mappati correttamente e che l'ordine degli array venga rispettato durante la conversione. Task 21.2: Creare una classe utility dedicata esclusivamente alla lettura e scrittura dei file fisici sul dispositivo. Task 21.3: Creare il controller che aggrega il catalogo tracce e il catalogo playlist, li serializza tramite il Task 1 e li passa al servizio di I/O del Task 2.	5

		Task 21.4: Intercettare l'avvio dell'applicazione per caricare i dati dal file JSON e popolare i cataloghi in memoria. Task 21.5: Gestire i possibili fallimenti durante le operazioni di I/O (es. permessi negati, disco pieno) e mostrare i relativi messaggi all'utente.	
--	--	---	--

Update Product Backlog

ID	TITLE	ITEM	PRIORITY	SP	ACCEPTANCE CRITERIA	STATUS
1	Aggiunta traccia al catalogo	Come utente voglio aggiungere un nuovo brano al catalogo inserendo titolo, autore, durata, genere e anno di pubblicazione, così da costruire la mia collezione musicale.	1	3	AC1 - Visualizzazione form AC2 - Campi obbligatori AC3 - Formato durata AC4 - Durata valida AC5 - Traccia duplicata	Fatto
2	Visualizzazione catalogo	Come utente voglio visualizzare tutte le tracce salvate nel catalogo con i loro attributi, così da avere sempre una visione completa della mia collezione.	1	2	AC1 - Visualizzazione tracce AC2 - Attributi visibili AC3 - Catalogo vuoto	Fatto
3	Visualizzazione catalogo playlist	Come utente voglio visualizzare le playlist salvate con le tracce all'interno.	1	2	AC1 - Visualizzazione playlist AC2 - Tracce nella playlist AC3 - Catalogo playlist vuoto	Fatto
4	Modifica traccia	Come utente voglio modificare i dati di un brano esistente, così da mantenere il catalogo sempre aggiornato.	1	2	AC1 - Apertura form precompilato AC2 - Salvataggio modifiche AC3 - Annulla modifiche AC4 - Campi obbligatori AC5 - Formato durata	Fatto

					AC6 - Nessuna traccia selezionata	
5	Eliminazione traccia	Come utente voglio eliminare una traccia dal catalogo, così da rimuovere contenuti che non voglio più.	1	3	AC1 - Selezione e eliminazione AC2 - Conferma eliminazione AC3 - Annulla eliminazione AC4 - Propagazione alle playlist AC5 - Nessuna traccia selezionata AC6 - Catalogo vuoto dopo eliminazione	Fatto
6	Creazione playlist	Come utente voglio creare una nuova playlist assegnandole un nome univoco, così da organizzare i miei brani in raccolte tematiche.	1	2	AC1 - Inserimento nome AC2 - Nome vuoto AC3 - Nome duplicato AC4 - Creazione corretta	Fatto
7	Aggiunta brani alla playlist	Come utente voglio aggiungere brani a una playlist, così da popolare le mie raccolte musicali.	1	3	AC1 - Selezione traccia AC2 - Aggiunta alla playlist AC3 - Duplicati permessi AC4 - Stessa traccia in playlist diverse AC5 - Selezione multipla AC6 - Nessuna playlist disponibile	Fatto
8	Rimozione brani dalla playlist	Come utente voglio rimuovere una traccia da una playlist	1	2	AC1 - Selezione e rimozione AC2 - Traccia non eliminata dalla libreria	Fatto

		senza eliminarla dal catalogo.				
9	Controlli singola traccia	Come utente voglio fare play, pausa e skip di una singola traccia.	1	5	AC1 - Avvio riproduzione AC2 - Pausa AC3 - Riprendi dopo pausa AC4 - Traccia successiva AC5 - Traccia precedente AC6 - Fine traccia automatica AC7 - Ultima traccia AC8 - Play senza traccia selezionata AC9 - Precedente sulla prima traccia	Fatto
10	Modalità di riproduzione playlist	Come utente voglio scegliere la modalità di riproduzione di una playlist tra sequenziale, loop e shuffle.	1	5	AC1 - Selezione modalità AC2 - Modalità Sequenziale AC3 - Modalità Loop AC4 - Modalità Shuffle	Fatto
11	Modalità di riproduzione singola traccia	Come utente voglio scegliere la modalità di riproduzione di una singola traccia tra sequenziale, loop e shuffle.	1	5	AC1 - Selezione modalità AC2 - Modalità Sequenziale AC3 - Modalità Loop AC4 - Modalità Shuffle	Fatto
12	Modifica playlist in esecuzione	Come utente voglio aggiungere o rimuovere tracce da una playlist mentre è in riproduzione.	1	5	AC1 - Aggiunta traccia senza interruzione AC2 - Rimozione traccia senza interruzione AC3 - Rimozione	Fatto

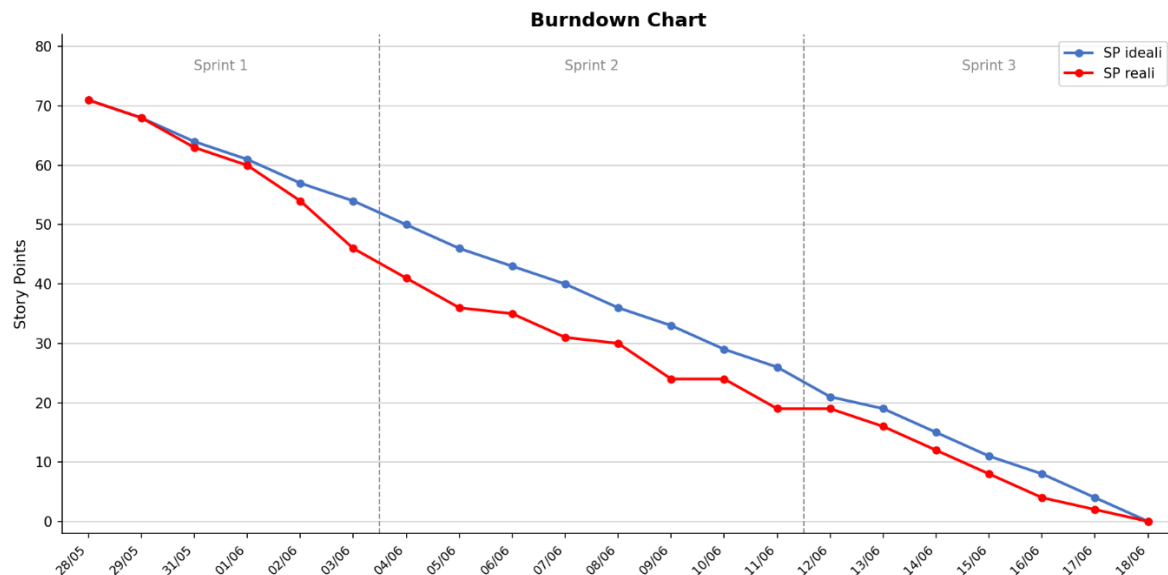
					traccia in esecuzione AC4 - Traccia aggiunta in coda	
13	Aggiornamento UI in tempo reale	Come utente voglio vedere il brano corrente, l'autore, il tempo trascorso e una barra di progresso aggiornati in tempo reale.	1	5	AC1 - Nome e autore visibili AC2 - Tempo trascorso e durata totale AC3 - Barra di progresso	Fatto
14	Undo operazioni	Come utente voglio annullare l'ultima operazione di aggiunta o rimozione di una traccia da una playlist.	2	3	AC1 - Pulsante Annulla disponibile AC2 - Undo aggiunta alla libreria AC3 - Undo rimozione dalla libreria AC4 - Undo aggiunta a una playlist AC5 - Undo rimozione da una playlist AC6 - Pulsante disabilitato	Fatto
15	Statistiche home page	Come utente voglio vedere nella home page le tracce e le playlist riprodotte più frequentemente, così da ritrovare subito i miei contenuti preferiti.	2	3	AC1 - Sezione tracce più riprodotte AC2 - Sezione playlist più riprodotte AC3 - Ordinamento per riproduzioni AC4 - Aggiornamento automatico	Fatto
16	Playlist automatica per genere e anno	Come utente voglio generare automaticamente una playlist con tutte le tracce di	2	5	AC1 - Generazione per genere/anno	Fatto

		un determinato genere/anno.				
17	Aggiunta tag visivi	Come utente voglio aggiungere tag visivi (Preferito, Esplicito, Nuova Uscita) a una traccia.	2	3	AC1 - Tag disponibili AC2 - Attivazione e disattivazione tag AC3 - Tag multipli AC4 - Tag visibili nella lista AC5 - Persistenza tag	Fatto
18	Playlist automatica per tag	Come utente voglio generare automaticamente una playlist per tag visivo con le tracce corrispondenti, così da organizzare rapidamente i brani contrassegnati.	2	3	AC1 - Generazione automatica playlist per tag AC2 - Eliminazione automatica playlist vuota	Fatto
19	Riordino tracce nella playlist	Come utente voglio poter cambiare l'ordine delle tracce all'interno di una playlist affinché io possa personalizzare la sequenza di ascolto.	3	3	AC1 - Spostamento traccia AC2 - Aggiornamento UI immediato AC3 - Riordino durante riproduzione	Fatto
20	Rimozione playlist da CatalogoPlaylist	Come utente voglio eliminare una playlist dal catalogo playlist, così da rimuovere raccolte che non voglio più mantenere.	3	2	AC1 - Selezione e rimozione playlist AC2 - Conferma eliminazione playlist AC3 - Annulla eliminazione playlist	Fatto

					AC4 - Tracce non eliminate dal catalogo AC5 - Nessuna playlist selezionata AC6 - Catalogo playlist vuoto dopo eliminazione	
21	Salvataggio di tracce e playlist	Come utente voglio salvare tracce e playlist, così da mantenere i dati inseriti anche dopo la chiusura dell'applicazione.	3	5	AC1 - Salvataggio del catalogo tracce AC2 - Salvataggio degli attributi delle tracce AC3 - Salvataggio delle playlist AC4 - Salvataggio delle tracce nelle playlist AC5 - Mantenimento dell'ordine delle tracce AC6 - Ripristino dei dati salvati AC7 - Salvataggio con cataloghi vuoti AC8 - Errore durante il salvataggio	Fatto

Project Burndown Chart

Il grafico sottostante mostra l'evoluzione giornaliera del lavoro rimanente, misurato in Story Points (SP), rispetto al tempo disponibile. L'obiettivo principale è quello di fornire al team una rappresentazione visiva immediata dello stato di avanzamento del progetto, evidenziando se il lavoro sta procedendo secondo i piani o se si stanno accumulando ritardi.



In questa terza e ultima sprint, il team ha completato le 6 US pianificate, portando a zero gli Story Points residui e completando l'intero Product Backlog pianificato. Al termine della Sprint 3 (18/06/2026), tutte le user story previste risultano nello stato "Fatto".

Analisi del Burndown Chart

Analizzando il Burndown Chart del progetto, si osserva che alla fine della terza sprint le stime ideali e quelle effettive risultano allineate: la curva degli SP reali segue la curva ideale, confermando che il team ha completato i task nei tempi previsti. Con il completamento della Sprint 3, il totale degli SP residui è sceso a zero, indicando il completamento dell'intero Product Backlog pianificato. La velocity del team è rimasta stabile rispetto alle sprint precedenti.

Sprint Review Report

Sprint n.º[3] — [12/06/26] / [18/06/26]

Nel corso della terza e ultima Sprint, svoltasi dal 12 al 18 giugno 2026, il team ha completato con successo tutti gli obiettivi prefissati. Nello specifico, sono state portate a termine 6 US su 6 pianificate: US-14, US-15, US-18, US-19, US-20 e US-21, corrispondenti all'intero backlog della Sprint 3, per un totale di 19 Story Points.

Stato delle user stories

ID	ITEM	STATO FINALE	SP
14	Come utente voglio annullare l'ultima operazione di aggiunta o rimozione di una traccia da una playlist.	Fatto	3
15	Come utente voglio vedere nella home page le tracce e le playlist riprodotte più frequentemente, così da ritrovare subito i miei contenuti preferiti.	Fatto	3
18	Come utente voglio generare automaticamente una playlist per tag visivo con le tracce corrispondenti, così da organizzare rapidamente i brani contrassegnati.	Fatto	3
19	Come utente voglio poter cambiare l'ordine delle tracce all'interno di una playlist affinché io possa personalizzare la sequenza di ascolto.	Fatto	3
20	Come utente voglio eliminare una playlist dal catalogo playlist, così da rimuovere raccolte che non voglio più mantenere.	Fatto	2
21	Come utente voglio salvare tracce e playlist, così da mantenere i dati inseriti anche dopo la chiusura dell'applicazione.	Fatto	5

Sprint Retrospective Report

Al termine di questa 3rd Sprint, abbiamo organizzato una retrospective per riflettere sul lavoro svolto e valutare cosa ha funzionato.

● Stop Doing

Cosa smettere di fare perché ha avuto un impatto negativo:

Fare merge su main senza code review approvata, anche "solo per una piccola fix"

● Less Of

Cosa ridurre perché ha consumato risorse senza portare reale valore:

Tempo speso su dettagli grafici non prioritari

● Keep Doing

Cosa ha funzionato bene e deve essere mantenuto:

Testing prima del merge

● More of

Cosa ha avuto un impatto positivo e dovrebbe essere fatto più spesso:

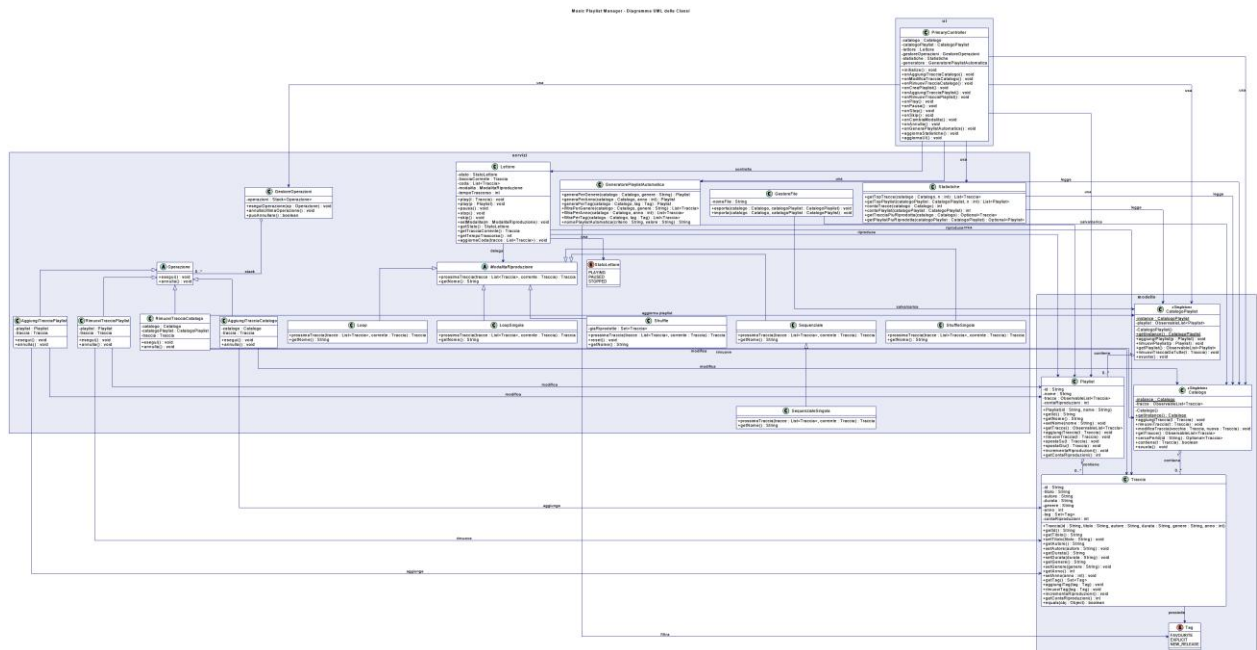
Pair programming su task complessi o rischiosi

● Start Doing

Cosa non è stato fatto ma che potrebbe portarci benefici:

Durante i test della GUI tenere traccia dei problemi riscontrati su un canale dedicato per sottoporli all'attenzione del team

Diagramma UML aggiornato



Il diagramma delle classi rappresenta l'architettura finale dell'applicazione Music Playlist Manager al termine della terza e ultima Sprint. Il sistema è organizzato in tre package distinti: **modello**, che raccoglie le entità dati (Traccia, Playlist, Catalogo, CatalogoPlaylist); **servizi**, che contiene la logica di business (Lettore, ModalitaRiproduzione e le sue sei implementazioni concrete, GestoreOperazioni, GestoreFile, Statistiche, GeneratorePlaylistAutomatica); e **ui**, che espone il PrimaryController come punto di accesso all'interfaccia grafica.

Rispetto alle sprint precedenti, il diagramma è stato arricchito con le seguenti aggiunte: le classi **SequenzialeSingola**, **LoopSingola** e **ShuffleSingola**, integrate nel Pattern Strategy per distinguere le modalità di riproduzione della singola traccia da quelle dell'intera playlist; i metodi **spostaSu()** e **spostaGiu()** nella classe Playlist, a supporto del riordino manuale delle tracce; e la classe **GeneratorePlaylistAutomatica** per la generazione automatica di playlist basata sui tag visivi, genere o anno. Il Pattern Command, già presente, è stato completato con la piena integrazione nel GestoreOperazioni per il supporto alla funzionalità di Undo.

Funzionalità Aggiuntive

Nel corso dello sviluppo del progetto, il team ha deciso autonomamente di implementare due funzionalità non previste inizialmente nel Product Backlog, ritenute necessarie per garantire un'esperienza utente completa e un comportamento corretto dell'applicazione.

Salvataggio su file JSON (US-21)

Il team ha scelto di implementare un meccanismo di persistenza dei dati basato su file JSON, gestito dalla classe `GestoreFile` tramite i metodi `esporta()` e `importa()`. Il salvataggio avviene in modo automatico ogni volta che il catalogo o le playlist vengono modificati, grazie a listener registrati sulle `ObservableList`. Le tracce all'interno delle playlist vengono salvate come riferimenti agli ID e non duplicate, garantendo la coerenza dei dati al ricaricamento. Questa funzionalità è stata ritenuta indispensabile per rendere l'applicazione realmente utilizzabile, evitando la perdita di tutti i dati ad ogni chiusura.

Eliminazione playlist dal catalogo (US-20)

Il team ha deciso di aggiungere la possibilità di eliminare una playlist dal catalogo, funzionalità non presente nei requisiti originali ma considerata fondamentale per una corretta gestione delle raccolte musicali. L'operazione è stata integrata nel bottone "Rimuovi" già esistente, rendendolo contestuale: rimuove una traccia se selezionata, oppure elimina l'intera playlist se nessuna traccia è selezionata. Prima della rimozione viene mostrato un dialog di conferma, e le tracce contenute nella playlist rimangono intatte nel catalogo generale.

Principi di Buon Design e Pattern Adottati

Nel corso dello sviluppo dell'applicazione Music Playlist Manager, il team ha adottato diversi principi e pattern di progettazione orientati a garantire un codice manutenibile, estensibile e ben strutturato. Di seguito vengono illustrati i principali principi applicati.

Pattern Strategy - ModalitaRiproduzione

Le modalità di riproduzione (Sequenziale, Loop, Shuffle e le rispettive varianti Singola) sono state implementate seguendo il Pattern Strategy. Ogni modalità è incapsulata in una classe concreta che estende la classe astratta ModalitaRiproduzione, la quale definisce il metodo prossimaTraccia(). Il Lettore delega il calcolo della traccia successiva alla modalità corrente tramite setModalita(), rendendo le strategie intercambiabili a runtime senza modificare il lettore stesso.

Pattern Singleton - Catalogo e CatalogoPlaylist

Le classi Catalogo e CatalogoPlaylist sono implementate come Singleton, garantendo un'unica istanza condivisa in tutta l'applicazione. Questo assicura la coerenza dei dati tra i vari componenti e semplifica l'accesso al modello senza necessità di passare riferimenti espliciti tra i controller.

Pattern Command - Operazione e GestoreOperazioni

Ogni operazione di modifica del catalogo o delle playlist è incapsulata in una classe concreta che estende la classe astratta Operazione, con i metodi esegui() e annulla(). Il GestoreOperazioni mantiene uno stack delle operazioni eseguite, abilitando la funzionalità di Undo in modo pulito e senza accoppiamento diretto tra UI e logica di business.

Pattern Observer - ObservableList

L'utilizzo delle ObservableList di JavaFX consente alle componenti grafiche (TableView, ListView) di aggiornarsi automaticamente al variare dei dati sottostanti, senza necessità di aggiornamenti manuali. Questo principio, ispirato al Pattern Observer, garantisce la sincronizzazione immediata tra modello e vista.

Separation of Concerns

Il codice è organizzato in tre package distinti con responsabilità ben separate: modello per le entità dati (Traccia, Playlist, Catalogo), servizi per la logica di business (Lettore, GestoreFile, GeneratorePlaylistAutomatica), e ui per la gestione dell'interfaccia grafica (PrimaryController). Questa separazione facilita la manutenibilità e la testabilità del codice.

DRY - Don't Repeat Yourself

Il principio DRY è stato applicato nella gerarchia delle modalità di riproduzione: SequenzialeSingola estende Sequenziale invece di duplicarne la logica, sovrascrivendo solo il comportamento diverso all'ultima traccia. Analogamente, la logica comune tra le varie modalità è centralizzata nella classe astratta ModalitaRiproduzione.

Single Responsibility Principle

Ogni classe ha una responsabilità ben definita e circoscritta: GestoreFile si occupa esclusivamente del salvataggio e caricamento dei dati, Statistiche gestisce solo il calcolo delle statistiche di ascolto, GeneratorePlaylistAutomatica si occupa unicamente della generazione automatica delle playlist. Questo principio riduce l'accoppiamento e semplifica la comprensione e la modifica del codice.

YAGNI - You Aren't Gonna Need It

Il principio YAGNI è stato rispettato durante l'intero sviluppo dell'applicazione, evitando di implementare funzionalità, metodi o classi non strettamente necessari ai requisiti correnti. Ogni componente del sistema è stato progettato per soddisfare esclusivamente i casi d'uso definiti nelle user story, senza aggiungere codice speculativo pensato per possibili evoluzioni future.

Ad esempio, la classe Lettore espone solo i metodi effettivamente utilizzati dal PrimaryController (play, pausa, stop, skip, setModalita), senza implementare funzionalità aggiuntive come code prioritarie, gestione di playlist multiple simultanee o sistemi di raccomandazione che avrebbero aumentato inutilmente la complessità del codice. Analogamente, GestoreOperazioni implementa solo la funzionalità di Undo tramite uno stack, senza aggiungere il Redo, non richiesto da nessuna user story. La classe Statistiche calcola esclusivamente le top tracce e le top playlist per numero di riproduzioni, senza introdurre analisi avanzate non previste nei requisiti. Questo approccio ha permesso al team di mantenere il codice snello, comprensibile e focalizzato sugli obiettivi reali del progetto, riducendo il rischio di introdurre complessità non necessaria e facilitando la manutenzione nel tempo.

